1. Explain the differences between software quality control and software quality assurance.

Software Quality Control (QC) refers to **a set of activities aimed at evaluating and verifying the quality of the final product.** It typically occurs after development, focusing on identifying defects through testing and inspection before the product is delivered to the client.

Software Quality Assurance (QA) is a broader process that spans the entire software development lifecycle. Its goal is to **prevent defects** by improving development processes, enforcing standards, and **detecting errors early** to reduce the cost and rate of defective products.

While **both aim to ensure product quality**, QA is proactive and **process-oriented**, whereas QC is reactive and **product-focused**. Importantly, quality control is considered a subset of quality assurance.

2. List the structures (categories and factors) for McCall's classic factors model.
   - Product operation factors
   - Product revision factors
   - Product transition factors

3. What are the differences (in terms of structures) between McCall's classic factor models, Evans and Marciniak factor model and Deutsch and Willis factor model? List the differences.

Answer:

Software Quality Factors

Other Factors Model

No.	Software quality factor	McCall's classic model	Alternative factor models	
			Evans and Marciniak	Deutsch and Willis
1	Correctness	+	+	+
2	Reliability	+	+	+
3	Efficiency	+	+	+
4	Integrity	+	+	+
5	Usability	+	+	+
6	Maintainability	+	+	+
7	Flexibility	+	+	+
8	Testability	+		
9	Portability	+	+	+
10	Reusability	+	+	+
11	Interoperability	+	+	+
12	Verifiability		+	+
13	Expandability		+	+
14	Safety			+
15	Manageability			+
16	Survivability			+

4. The software requirement document for the tender for development of "Super-lab", a software system for managing a hospital laboratory, consists of chapters according to the required quality factors as follows: correctness, reliability, efficiency, integrity, usability, maintainability, flexibility, testability, portability, reusability and interoperability.

In the following table you will find sections taken from the mentioned requirements document. For each section, fill in the name of the factor that best fits the requirement (choose only one factor per requirements section).

No	Section taken from software requirements document	Requirement Factor
1	The probability that the "Super-lab" software system will be found in a state of failure during peak hours (9 am to 4 pm) is required to be below 0.5%.	Reliability
2	The "Super-lab" software system will enable direct transfer of laboratory results to those files of hospitalized patients managed by the "MD-File" software package.	Flexibility
3	The "Super-lab" software system will include a module that prepares a detailed report of the patient's laboratory test results during his or her current hospitalization. (This report will serve as an appendix to the family physician's file.) The time required to obtain this printed report will be less than 60 seconds; the level of accuracy and completeness will be at least 99%.	Correctness
4	The "Super-lab" software to be developed for hospital laboratory use may be adapted later for private laboratory use.	Flexibility
5	The training of a laboratory technician, requiring no more than three days, will enable the technician to reach level C of "Super-lab" software usage. This means that he or she will be able to manage reception of 20 patients per hour.	Usability
6	The "Super-lab" software system will record a detailed users' log. In addition, the system will report attempts by	Integrity

	unauthorized persons to obtain medical information from the laboratory test results database. The report will include the following information: network identification of the applying terminal, system code of the employee who requested that information, day and time of attempt, and type of attempt.	
7	The "Super-lab" subsystem that deals with billing patients for their tests may eventually be used as a subsystem in the "Physiotherapy Center" software package.	Reusability
8	The "Super-lab" software system will process all the monthly reports for the hospital departments' management, the hospital management, and the hospital controller according to Appendix D of the development contract.	Correctness
9	The software system should be able to serve 12 workstations and eight automatic testing machines with a single model AS20 server and a CS25 communication server that will be able to serve 25 communication lines. This hardware system should conform to all availability requirements as listed in Appendix C.	Efficiency
10	The "Super-lab" software package developed for the Linux operating system should be compatible for applications in a Windows NT environment.	Portability

5. List and explain the components of SQA system.

1. Pre-project components
    - To assure that:
        - The project commitments have been adequately defined considering the resources required, the schedule and budget.
        - The development and quality plans have been correctly determined.
    - Improve the preparatory steps to initiate works on the project:
        - Contract review
            - Detailed examination of (a) the project proposal draft and (b) the contract drafts.
            - Development unit is committed to an agreed-upon functional specification, budget and schedule.
        - Development and quality plans
            - Prepare the plan for project ("development plan" – schedule, resources, risk evaluation, etc.) and its integrated quality assurance activities ("quality plan" – quality goals, list of review and test, etc.).

2. Software Project Life Cycle Components
    - Composed of two stages:
        - Development life cycle stage
            - To detect design and programming errors
            - Using review, expert opinion and software testing
        - Operation-maintenance stage
            - Specialized maintenance components, applied for functionality improving maintenance tasks.

3. Infrastructure Components for Error Prevention and Improvement
    - Prevention of software faults or lowering of software fault rates, together with the improvement of productivity.
    - SQA components here including:
        - Procedures and work instructions
        - Templates and checklists

        - Staff training, retraining, and certification
        - Preventive and corrective actions
        - Configuration management
        - Documentation control.

4. Management SQA Components
    - The control of development and maintenance activities and the introduction of early managerial support actions that mainly prevent or minimize schedule and budget failures and their outcomes.
    - Support the managerial control of software development projects and maintenance services.
    - Control components including:
        - Project progress control (including maintenance contract control)
        - Software quality metrics
        - Software quality costs.

5. SQA standards, system certification, and assessment components
    - To implement international professional and managerial standards within the organization.
    - Objectives:
        - Utilization of international professional knowledge
        - Improvement of coordination of the organizational quality systems with other organizations
        - Assessment of the achievements of quality systems according to a common scale
    - Standards include quality management standards and project process standards.

6. Human components
    - Human components are the SQA organizational base includes managers, testing personnel, the SQA unit and practitioners (SQA trustees, SQA committee members and SQA forum members).
    - Objectives of SQA organizational base:

- To develop and support implementation of SQA components.
- To detect deviations from SQA procedures and methodology.
- To suggest improvements to SQA components.

6. List and explain the SQA activities in Software Testing

1. Plan the testing and SQA processes
    - Test processes should be well planned, defined, and documented.
        i. Quality management plan
            - Defines an acceptable level of product quality and describes how the project will achieve this level
        ii. Test plan
            - Describes what to test, when to test, how to test, and who will do the tests
        iii. Test cases
            - Consists of a set of actions which are performed on the software application in order to verify the expected functionality of the feature.

2. Implementation of test-oriented management
    - Using Extreme Programming (EX) software development methodology.
    - Aims to produce higher quality software with the ability to adapt to changing requirements.
    - Test-driven development:
        i. Tests are written before any implementation of code (Test-first Approach)
        ii. New feature begins with writing a test - automated test case before writes enough code to fulfill
        iii. This test case will initially fail, next will be to write the code focusing on functionality to get that test passed.
        iv. After these steps are completed, refactors the code to pass all the tests.
    - Pair programming:
        i. Requires two developers working in cycle at a single computer.

ii. One of them writes a code while the other watches and makes suggestions through the process. These roles can be swapped at any time.

iii. Will produce software with a significantly higher quality, reducing the debugging and refactoring cost of the project in the long run.

3. Ensure suitable work environment for SQA team
    - Involve the dedicated SQA team from the beginning to start testing early to be assured that testing is done professionally.
    - Respect the testers by build trusting relationships between a SQA team and developers with respect for each other.
    - Give business training to the SQA team to expand their knowledge and to improve the work of the entire team.
    - Importance of communication between testers and developers to avoid misunderstandings.

4. Optimize the use of automated tests
    - To improve the quality of a software, automated testing (using automation tools to run the tests) is definitely worth taking into consideration.
    - Two of three key trends are increasing test automation and widespread adoption of the Agile methodologies.
    - It's really a wise recommendation to deploy automated testing throughout the QA process.

5. Employ code quality measurements
    - Quality objectives should be measurable, documented, reviewed, and tracked.
    - The best advice is to choose metrics which are simple and effective for the workflow.
    - The Consortium for IT Software Quality (CISQ) software quality model defines four important aspects of software quality:
        i. Reliability
            - Risk of software failure due to unexpected conditions.
            - Reliable software; minimal downtime, good data integrity, no errors that directly affect users.

ii. Performance Efficiency
- Application's use of resources that affects its scalability, customer satisfaction, response times.

iii. Security
- How well an application protects information.
- The quantity and severity of vulnerabilities indicate its security level.

iv. Maintainability
- Ease in modify software, adapt for other purposes, or transfer it from one development team to another.

- The model can be expanded to include the assessment:

i. Rate of Delivery
- How often new versions of software are shipped to customers.
- Higher rates of delivery correspond to better quality software for customers.

ii. Testability
- Quality software requires a high degree of testability.
- Finding faults in software with high testability is easier, making such systems less likely to contain errors when shipped to end users.

iii. Product Usability
- The UI is the only part of the software visible to users, so it's vital to have a good UI.

6. Report bugs effectively
   - A good bug report will help:
     i. Make software testing more efficient by
     ii. Clearly identifying the problem
     iii. Navigating engineers towards solving it
   - A badly written report can lead to serious misunderstanding.

7. Discuss the benefits of applying SQA activities.

- **Early Defect Detection**

    - Reduces rework and cost by catching issues early in the development process.
- **Improved Product Quality**
    - Ensures the software meets customer expectations and standards.
- **Better Process Control**
    - Encourages disciplined, repeatable practices that improve long-term productivity.
- **Customer Satisfaction**
    - Delivers a reliable, functional product, increasing client trust and confidence.
- **Compliance and Documentation**
    - Supports industry standards and legal/regulatory compliance.

8. Discuss the challenges in applying SQA activities.

- **Lack of Resources**
    - Limited time, budget, or skilled personnel can hinder quality efforts.
- **Resistance to Process Change**
    - Teams may resist adopting formal QA procedures or standards.
- **Incomplete Requirements**
    - Poorly defined requirements can make it difficult to create effective tests.
- **Tool Integration Issues**
    - Challenges in integrating QA tools with existing systems.
- **Management Support**
    - Lack of commitment from management can limit the success of SQA initiatives.